WEEK-3 BUSINESS ANALYTICS

Multi-Echelon Supply Chain Network Design via Mixed-Integer Linear Programming (MILP)

Student Details

Name ; Rajesh . S

Dept ; BSC.,CS(AIML) 2nd year

Enroll No ; TU - 6235210211044

1. Task Name

Multi-Echelon Supply Chain Network Design via Mixed-Integer Linear Programming (MILP)

2. Description

Students will model a global logistics network with factories, warehouses, and customer zones. They must write formal linear programming constraint matrices representing multi-echelon transport limits, warehouse storage capacities, and minimum service level bounds, solving the optimization problem to minimize total supply chain operating expenditures.

3. What is MILP?

MILP stands for Mixed-Integer Linear Programming. It is an optimization technique used when some decisions are continuous and some decisions must be integer or binary.

Examples:

- How many units should each factory produce?
- How many units should be transported from factories to warehouses?
- How many units should be transported from warehouses to customers?
- Should a warehouse be opened?
- What is the minimum total supply-chain cost?

4. Supply Chain Structure

```
Factory 1 ──────► Warehouse 1 ──────► Customer Zone 1
Factory 2 ──────► Warehouse 2 ──────► Customer Zone 2
Factory 3 ──────► Warehouse 3 ──────► Customer Zone 3
```

```
The objective is to find the cheapest way to transport products from factories
to customers while respecting factory capacity, warehouse capacity,
transportation limits, and customer service requirements.
```

5. Sample Factory Data

Factory	Capacity
F1	500
F2	600
F3	400

6. Sample Warehouse Data

Warehouse	Capacity	Fixed Cost
W1	500	₹20,000
W2	600	₹25,000
W3	400	₹18,000

7. Sample Customer Zone Data

Customer	Demand
C1	300
C2	400
C3	350
C4	250

Total demand = 1,300 units.

8. Decision Variables

x_fw = Number of units transported from factory f to warehouse w.

y_wc = Number of units transported from warehouse w to customer c.

z_w = Binary warehouse opening variable.

z_w = 1 → Warehouse is opened
z_w = 0 → Warehouse is closed

9. Objective Function

Minimize:

$$Z = \Sigma(C_{fw} X_{fw}) + \Sigma(C_{wc} Y_{wc}) + \Sigma(F_w Z_w)$$

Where:
C_fw = Factory-to-Warehouse transportation cost
X_fw = Factory-to-Warehouse quantity
C_wc = Warehouse-to-Customer transportation cost
Y_wc = Warehouse-to-Customer quantity
F_w = Warehouse fixed operating cost
Z_w = Warehouse open/close decision

10. Factory Capacity Constraint

A factory cannot produce or ship more than its capacity.

For Factory 1:
$X11 + X12 + X13 \leq 500$

For Factory 2:
$X21 + X22 + X23 \leq 600$

For Factory 3:
$X31 + X32 + X33 \leq 400$

General form:
$\Sigma X_{fw} \leq Capacity_f$

11. Warehouse Capacity Constraint

A warehouse cannot handle more products than its capacity.

General form:

$\Sigma X_{fw} \leq Cap_w Z_w$

If $Z_w = 1$, the warehouse can operate up to its capacity.
If $Z_w = 0$, the warehouse cannot be used.

12. Warehouse Flow Balance

Whatever enters a warehouse must be available for distribution to customers.

General form:

$\Sigma X_{fw} = \Sigma Y_{wc}$

This means:

Total incoming quantity = Total outgoing quantity.

13. Customer Demand Constraint

Every customer zone must receive its required demand.

Customer 1:
$Y11 + Y21 + Y31 = 300$

Customer 2:
$Y12 + Y22 + Y32 = 400$

Customer 3:
$Y13 + Y23 + Y33 = 350$

Customer 4:

$$Y_{14} + Y_{24} + Y_{34} = 250$$

General form:

$$\sum Y_{wc} = Demand_c$$

14. Minimum Service Level

Suppose the required minimum service level is 95%.

General form:

$$\sum Y_{wc} \geq 0.95 \times Demand_c$$

For example, for Customer 1:

$$300 \times 0.95 = 285 \text{ units}$$

Therefore, at least 285 units must be supplied to Customer 1 under a 95% service-level constraint.

15. Non-Negativity Constraints

Products cannot be transported in negative quantities.

$$X_{fw} \geq 0$$
$$Y_{wc} \geq 0$$

16. Binary Constraints

Warehouse opening decisions must be either 0 or 1.

$$Z_w \in \{0, 1\}$$

0 = Warehouse closed
1 = Warehouse opened

17. Complete Mathematical Model

Objective:

Minimize
$$Z = \sum(C_{fw} X_{fw}) + \sum(C_{wc} Y_{wc}) + \sum(F_w Z_w)$$

Subject to:

Factory capacity:
$$\sum X_{fw} \leq S_f$$

Warehouse capacity:
$$\sum X_{fw} \leq Cap_w Z_w$$

Warehouse flow balance:
$\Sigma X_{fw} = \Sigma Y_{wc}$

Customer demand:
$\Sigma Y_{wc} = D_c$

Minimum service level:
$\Sigma Y_{wc} \geq \alpha D_c$

Non-negativity:
$X_{fw}, Y_{wc} \geq 0$

Binary warehouse decision:
$Z_w \in \{0,1\}$

Where $\alpha = 0.95$ for a 95% minimum service level.

18. Constraint Matrix Concept

The constraint matrix represents the mathematical constraints in matrix form:

$A x \leq b$

Where:
A = Constraint coefficient matrix
x = Vector of decision variables
b = Right-hand-side values

Example:

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} X11 \\ X12 \\ X13 \\ X21 \\ X22 \\ X23 \end{bmatrix} \leq \begin{bmatrix} 500 \\ 600 \end{bmatrix}$$

This represents factory capacity constraints.

19. Python Implementation

Install PuLP:

```
pip install pulp
```

Import the library:

```
import pulp
```

Create the optimization model:

```
model = pulp.LpProblem(
    "Supply_Chain_Network_Design",
```

```
    pulp.LpMinimize
)
```

20. Define Sets

```
factories = ["F1", "F2", "F3"]
warehouses = ["W1", "W2", "W3"]
customers = ["C1", "C2", "C3", "C4"]
```

21. Define Capacities

```
factory_capacity = {
    "F1": 500,
    "F2": 600,
    "F3": 400
}

warehouse_capacity = {
    "W1": 500,
    "W2": 600,
    "W3": 400
}

demand = {
    "C1": 300,
    "C2": 400,
    "C3": 350,
    "C4": 250
}
```

22. Create Decision Variables

```
x = pulp.LpVariable.dicts(
    "Factory_Warehouse",
    [(f, w) for f in factories for w in warehouses],
    lowBound=0,
    cat="Continuous"
)

y = pulp.LpVariable.dicts(
    "Warehouse_Customer",
    [(w, c) for w in warehouses for c in customers],
    lowBound=0,
    cat="Continuous"
)

z = pulp.LpVariable.dicts(
    "Warehouse_Open",
    warehouses,
    cat="Binary"
)
```

23. Add Factory Constraints

```
for f in factories:
    model += (
        pulp.lpSum(x[(f, w)] for w in warehouses)
        <= factory_capacity[f]
    )
```

24. Add Warehouse Constraints

```
for w in warehouses:
    model += (
        pulp.lpSum(x[(f, w)] for f in factories)
        <= warehouse_capacity[w] * z[w]
    )
```

25. Add Flow Balance

```
for w in warehouses:
    model += (
        pulp.lpSum(x[(f, w)] for f in factories)
        ==
        pulp.lpSum(y[(w, c)] for c in customers)
    )
```

26. Add Customer Demand

```
for c in customers:
    model += (
        pulp.lpSum(y[(w, c)] for w in warehouses)
        == demand[c]
    )
```

27. Transportation Cost Data

```
factory_warehouse_cost = {
    ("F1", "W1"): 5,
    ("F1", "W2"): 7,
    ("F1", "W3"): 9,
    ("F2", "W1"): 6,
    ("F2", "W2"): 4,
    ("F2", "W3"): 8,
    ("F3", "W1"): 8,
    ("F3", "W2"): 6,
    ("F3", "W3"): 5
}

warehouse_customer_cost = {
    ("W1", "C1"): 4,
    ("W1", "C2"): 6,
    ("W1", "C3"): 7,
    ("W1", "C4"): 8,
```

```
    ("W2", "C1"): 6,
    ("W2", "C2"): 4,
    ("W2", "C3"): 5,
    ("W2", "C4"): 7,
    ("W3", "C1"): 8,
    ("W3", "C2"): 7,
    ("W3", "C3"): 4,
    ("W3", "C4"): 5
}

fixed_cost = {
    "W1": 20000,
    "W2": 25000,
    "W3": 18000
}
```

28. Objective Function in Python

```
model += (
    pulp.lpSum(
        factory_warehouse_cost[(f, w)] * x[(f, w)]
        for f in factories
        for w in warehouses
    )
    +
    pulp.lpSum(
        warehouse_customer_cost[(w, c)] * y[(w, c)]
        for w in warehouses
        for c in customers
    )
    +
    pulp.lpSum(
        fixed_cost[w] * z[w]
        for w in warehouses
    )
)
```

29. Solve the Model

```
model.solve()

print("Status:", pulp.LpStatus[model.status])
print("Minimum Cost:", pulp.value(model.objective))

for w in warehouses:
    print(w, "Open =", z[w].value())

for f in factories:
    for w in warehouses:
        if x[(f, w)].value() > 0:
            print(f, "->", w, "=", x[(f, w)].value())
```

30. Expected Business Analytics Output

Optimal Supply Chain Network

Open Warehouses:
W1
W2
W3

Factory → Warehouse:
F1 → W1 : xxx units
F1 → W2 : xxx units
F2 → W2 : xxx units
F3 → W3 : xxx units

Warehouse → Customer:
W1 → C1 : xxx units
W2 → C2 : xxx units
W3 → C3 : xxx units

Minimum Total Cost:
₹xxxxx

The exact quantities and cost are determined by the optimization solver.

31. Simple Explanation for Viva

This project uses Mixed-Integer Linear Programming to find the optimal supply-chain network. We have factories, warehouses, and customer zones. The model decides how much product should move from each factory to each warehouse and from each warehouse to each customer. It also decides which warehouses should be opened. The objective is to minimize total transportation and warehouse operating costs while satisfying capacity, demand, and minimum service-level constraints.

32. Key Concepts Learned

- Business Analytics
- Optimization
- Linear Programming
- Mixed-Integer Linear Programming (MILP)
- Decision Variables
- Objective Function
- Constraints

- Constraint Matrix
- Optimization Solver
- Optimal Supply Chain Network